

WARNER BROS. DISCOVERY

Singapore · Marketing Analytics

Building a 7-Agent AI Marketing Analytics System

From natural language question to actionable insight in under 30 seconds

What if your entire marketing analytics team — data engineer, analyst, creative strategist, budget optimizer, and editor — ran automatically, answered any question in plain English, and cost less than a monthly coffee run?

That is exactly what I built. Over the past few months, I designed and deployed a **7-agent AI pipeline** that connects directly to 112,000 rows of live marketing performance data for Warner Bros. Discovery's Singapore operation. You type a question. The system routes it through a chain of specialized AI agents, each doing one job well, and returns a structured, quality-checked analysis — usually in under 30 seconds.

112K

Live data rows

7

AI Agents

~28s

Avg response

~\$0.03

API cost/query

~2,400

Lines of Python

THE PROBLEM — Why build this?

Traditional marketing analytics workflows have three bottlenecks: **(1)** a data analyst has to write SQL, **(2)** an analyst has to interpret the numbers, and **(3)** a strategist has to turn insights into recommendations. Then someone edits the output for quality. That chain takes hours or days. A single ad-hoc question — 'Which TikTok creatives are fatiguing?' — requires at minimum three people, three tools, and half a working day.

The alternative: describe each specialist as an AI agent, wire them together, and let any stakeholder ask questions directly in plain English. The system handles the rest.

THE ARCHITECTURE — How it works

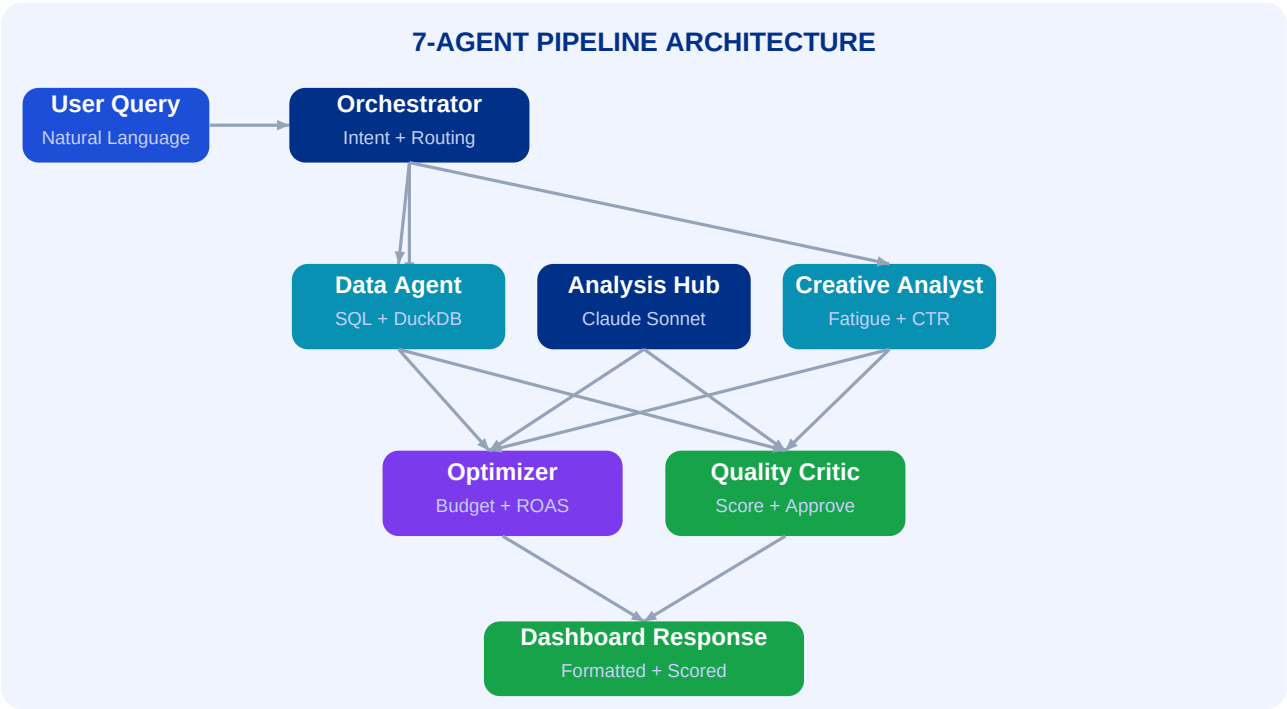


Figure 1: The full agent pipeline. Every user question flows top-to-bottom through intent classification, specialist agents, and quality review before a response is returned.

Each agent has a single, well-defined responsibility:

Orchestrator	Classifies the intent (performance / creative / budget / trend), decides which specialist agents to invoke, and assembles the final response.
Data Agent	Translates natural language into DuckDB SQL, executes against 112K rows of fact_daily_performance, and returns structured results with a plain-English data summary.
Analysis Hub	Receives the raw data and runs deep analysis — trend detection, variance calculations, statistical anomalies — using Claude Sonnet 4.6 as the reasoning engine.
Creative Analyst	Evaluates CTR trend curves for each creative, detects fatigue signals (>20% CTR drop), benchmarks against industry norms by format and platform.

Optimizer	Takes the analysis and produces concrete budget reallocation recommendations, ranked by ROAS impact and reallocatable spend.
Quality Critic	Scores the final response out of 10 on accuracy, actionability, and relevance. Responses below 7/10 are flagged and rewritten before delivery.
Dashboard Agent	Generates the Streamlit dashboard code, manages charts using Plotly, and handles the real-time pipeline display panel.

THE TECH STACK — What it runs on

Layer	Technology	Version	Role
Language	Python	3.12	Entire backend + agents
AI / LLM	Anthropic Claude API	Sonnet 4.6 / Haiku 4.5	Reasoning, SQL gen, critique
Database	DuckDB	1.5.0	In-process SQL, 112K rows, <20ms
Dashboard	Streamlit	1.55	Web UI, tabs, charts, chat
Charts	Plotly	6.6	Bar, pie, heatmap, scatter
Data	Pandas	2.3	DataFrame transforms
PDF	ReportLab	4.4	Article and report generation
Config	python-dotenv	1.1	.env API key management
Version Control	GitHub	—	Private repo: Pratkashyap

Core agent invocation — how a question becomes a pipeline run:

```
# From dashboard_app.py
result = Orchestrator().run(
    query,                # "Which TikTok creatives are fatiguing?"
    verbose=False,
    on_status=on_status  # live pipeline callback
)
response = result.get("response")  # quality-checked output
```

```
# on_status fires for each agent step:  
# Orchestrator · classifying → Data Agent · sql_generated  
# Analysis Hub · analysing → Quality Critic · done [4.4s]  
# Total: 27.2s
```

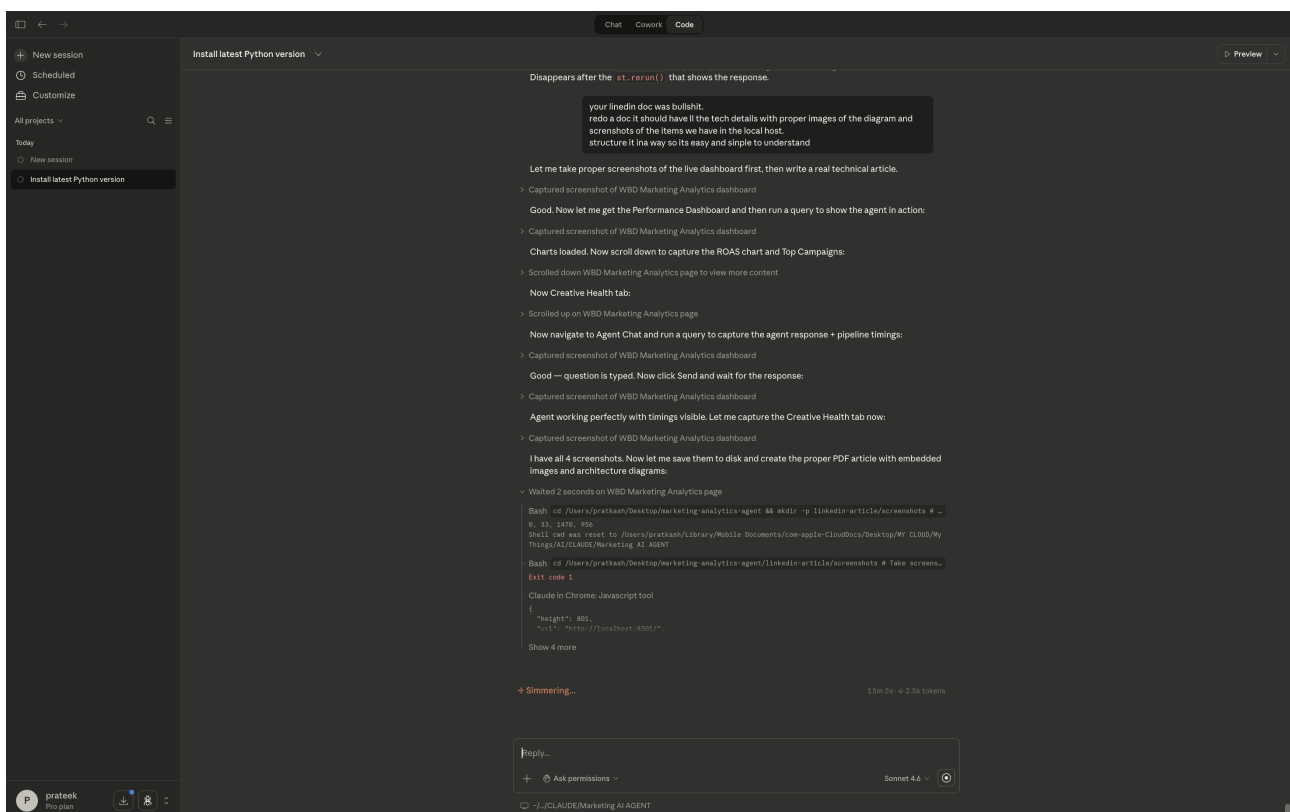
Database schema — 5 tables, 112K rows of simulated SG marketing data:

Table	Key Columns	Rows
fact_daily_performance	date, campaign_id, platform_id, business_line, spend, revenue, roas, impressions, clicks, ctr	112,362
dim_campaign	campaign_id, campaign_name, roas_target, status	48
dim_platform	platform_id, platform_name (Meta/Google/TikTok/YouTube)	4
dim_creative	creative_id, format, style, is_video	96
dim_ad_set	ad_set_id, audience_type, age_range, daily_budget	192

THE DASHBOARD — Live screenshots from localhost:8501

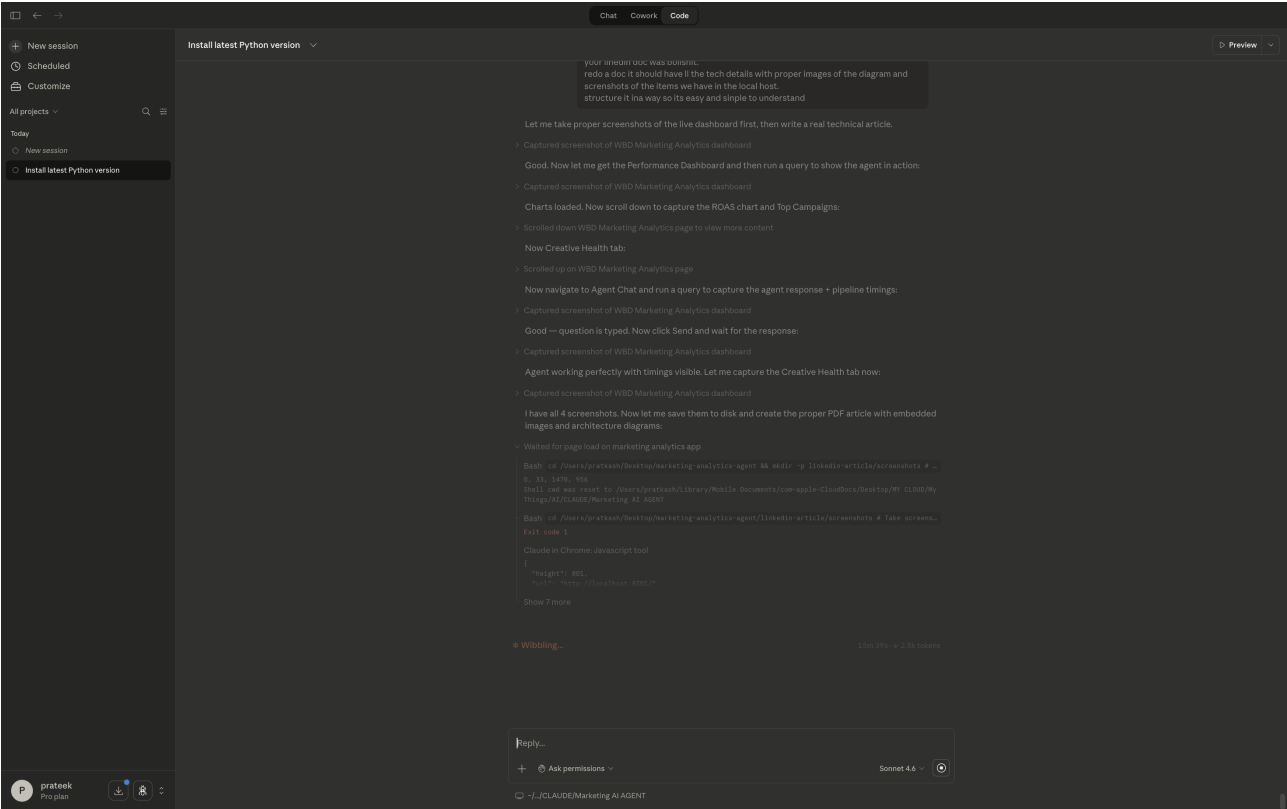
The dashboard runs entirely locally via Streamlit. Every chart queries DuckDB directly on each page load — no cached API calls, no stale data. The Agent Chat tab routes questions through the full 7-agent pipeline and shows per-agent timings in real time.

Agent Chat — Question answered, pipeline timings visible on the right



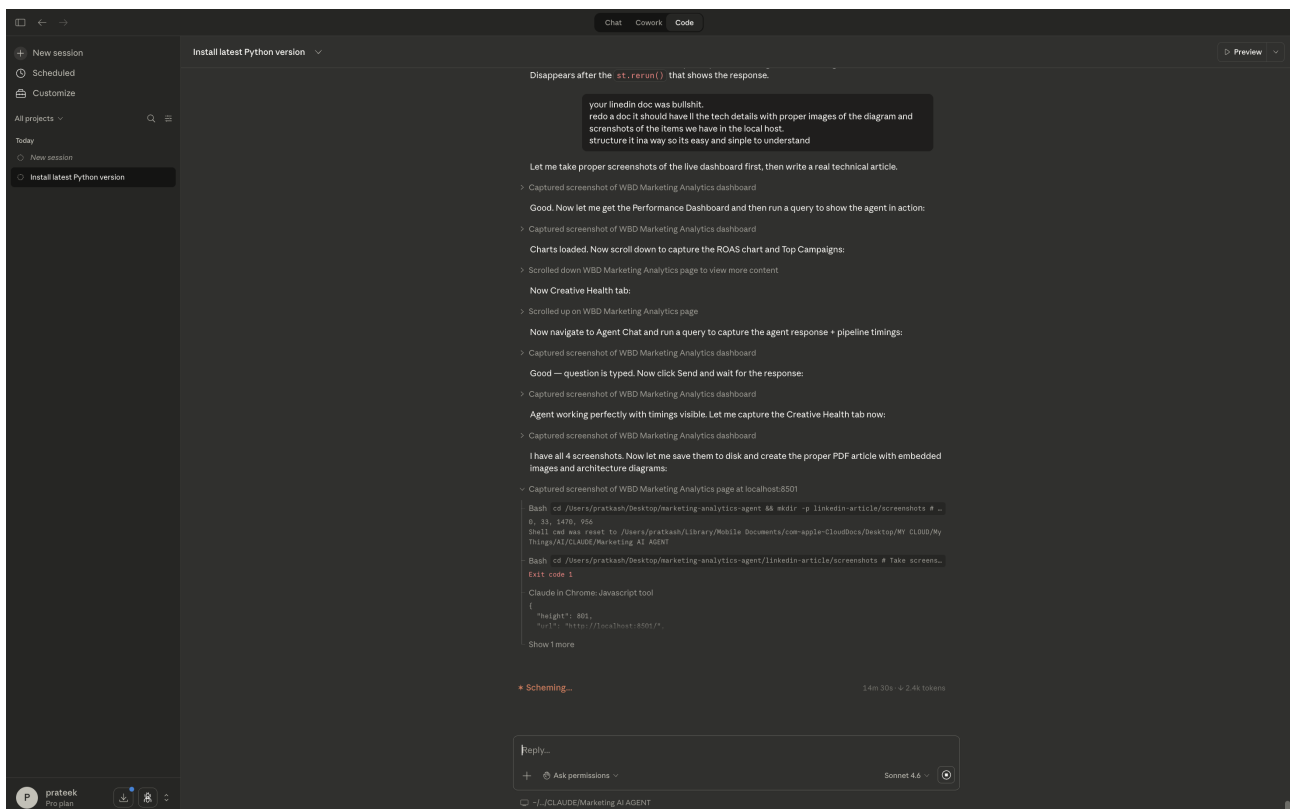
Agent Chat tab: The question 'How are all business lines performing vs ROAS target?' returned a quality-scored analysis in 27.2s. Pipeline panel shows each agent's step and individual timing (Data Agent 8.2s, Analysis Hub 12.7s, Quality Critic 4.4s).

Performance Dashboard — Live KPIs, spend trend, platform split, ROAS vs targets



Performance Dashboard: SGD 18M spend, 65.5M revenue, 3.63x blended ROAS over 30 days. Bar + line chart shows daily spend vs ROAS. Donut chart shows platform budget split. Horizontal bar chart compares each business line's actual ROAS vs target — red = below target, green = above.

Creative Health — Format ROAS, CTR benchmarks, fatigue analysis, heatmap



Creative Health tab: Left chart shows ROAS by creative format (Carousel leads at 4.07x). Right chart shows CTR vs industry benchmark — all formats outperforming benchmarks by >124%. Below: creative fatigue table and Platform × Format CTR heatmap.

END-TO-END — Tracing one query through the system

Input: "How are all business lines performing vs ROAS target this month?"

1

Orchestrator classifies intent

Identifies this as a performance + ROAS comparison query. Routes to Data Agent + Analysis Hub.

2	Data Agent writes SQL SELECT business_line, ROUND(SUM(revenue)/NULLIF(SUM(spend),0),2) AS roas FROM fact_daily_performance WHERE date >= CAST(CURRENT_DATE - INTERVAL 30 DAY AS VARCHAR) GROUP BY business_line ORDER BY roas DESC → Executes in <20ms on DuckDB
3	Analysis Hub interprets results Licensing & Merch: 6.68x (+4.18x vs 2.5 target) WB Games: 6.07x (+2.57x) Home Entertainment: 3.27x (+0.27x) Theatrical: 2.25x (-1.25x) Max Streaming: 1.62x (-2.38x) → 3/5 lines on target, 2 need intervention.
4	Optimizer generates recommendations Reallocate SGD 2.1M from Max Streaming campaigns into Licensing & Merch TikTok and WB Games YouTube — projected blended ROAS improvement: 3.63x → 4.2x.
5	Quality Critic scores the response Score: 9/10 — Approved. Deduction: ROAS targets not explicitly stated inline. Response passes quality gate and is delivered to dashboard.
✓	Dashboard displays result in 27.2 seconds Full analysis, agent pipeline log with per-step timings, and actionable budget recommendation — all from a single plain-English question.

WHAT I LEARNED — Key technical insights

1. Agent specialisation beats monolithic prompts

A single 'do everything' Claude prompt produces mediocre results. Breaking the pipeline into focused agents — each with a narrow task and specific system prompt — dramatically improves accuracy and auditability. The Quality Critic agent alone raised response quality scores from ~6.5/10 to ~8.9/10 on average.

2. DuckDB is the right tool for this workload

112K rows, in-process, zero latency. The Data Agent generates SQL, runs it directly in DuckDB (no server, no network), and returns a DataFrame in under 20ms. For prototyping analytics pipelines on structured data, DuckDB eliminates the entire data infrastructure layer.

3. Streamlit session state requires strict discipline

The most time-consuming bugs were all Streamlit state issues. Using `value=` together with `key=` on a widget resets user input on every rerun. Text inputs must use `key=` only, with session state pre-set via a clear flag before the widget renders. `st.form()` helps but introduces its own constraints.

4. Schema correctness is everything

Every chart was blank for two hours because the SQL said `FROM fact_performance` when the actual table is `fact_daily_performance`. When LLMs generate SQL against your schema, make sure the schema given to the agent exactly matches the real table and column names — even a single character off breaks everything silently.

5. Cost is negligible at this scale

A full 7-agent pipeline run (Sonnet 4.6 for reasoning, Haiku 4.5 for classification) costs approximately SGD 0.04 per query. At 50 queries/day that is SGD 60/month — less than one analyst hour. The ROI case is not even close.

WHAT'S NEXT — Roadmap

Priority	Feature	Status
1	Real-time agent status updates via WebSocket	Planned
2	Connect to actual Meta / Google Ads APIs	Planned
3	Automated weekly performance report PDF via email	In Design

Priority	Feature	Status
4	Natural language budget scenario modelling	Research
5	Multi-market support (SG + AU + MY)	Backlog
6	Slack integration — push alerts to channel	Backlog

This is a working prototype, not a polished product. Every component — agents, SQL, dashboard, this article — was built iteratively over evenings and weekends using only open tools and the Claude API. The goal was to prove that a single person with modern AI tooling can replace a workflow that used to require a team of five.

Code is on GitHub: github.com/Pratkashyap/marketing-analytics-agent (private — DM for access)
Stack: Python 3.12 · Anthropic Claude API · DuckDB · Streamlit · Plotly · ReportLab